Wallets & transaction UX, the way real users experience them.

CONNECT, SIGN, CONFIRM — WITHOUT LYING TO YOUR USER.

SESSION GOALS

Wire the official Wallet Adapter into a React app and design a signature flow that users trust.

Why wallets, not logins

- A wallet is not an account — it's a **key** under user control.
- The app never sees the private key. It can only **request signatures**.
- Every signature is a **cryptographic permission** — no password is involved.

> If your UI cannot explain **what** it is asking the user to sign in one sentence, you have already lost their trust.

Source: anza-xyz/wallet-adapter

The official React wiring

```
import { ConnectionProvider, WalletProvider }
  from "@solana/wallet-adapter-react";
import { WalletModalProvider }
  from "@solana/wallet-adapter-react-ui";
import { PhantomWalletAdapter, SolflareWalletAdapter }
  from "@solana/wallet-adapter-wallets";
import "@solana/wallet-adapter-react-ui/styles.css";

export function SolanaProvider({ children }) {
  const endpoint = process.env.NEXT_PUBLIC_RPC!;
  const wallets = [new PhantomWalletAdapter(), new SolflareWalletAdapter()];
  return (
    <ConnectionProvider endpoint={endpoint}>
      <WalletProvider wallets={wallets} autoConnect>
        <WalletModalProvider>{children}</WalletModalProvider>
      </WalletProvider>
    </ConnectionProvider>
  );
}
```

Three providers, wrapped once. That is the whole story.

The three providers, explained

CONNECTION

Exposes an RPC `Connection` object to the tree. Drives reads and writes.

WALLET

Owns the connected wallet, available wallets, autoConnect, and signing surfaces.

WALLET MODAL

Renders the UI for wallet selection — the classic "connect wallet" button.

> Swap your RPC by changing **one environment variable**. No code changes, no hardcoded URLs.

Every signature should feel like the app asking permission, not taking it.

Pre-signature: set expectations

Before calling `sendTransaction`, show the user:

- **Who** is paying — the connected wallet.
- **What** is happening — "Send 0.1 SOL to `4Nd1…k3s`".
- **How much** it costs — network fee + priority fee (if enabled).
- **Where** they can verify — a link to Solana Explorer **after** success.

> Consistency matters. Use the same copy pattern on every signing surface in your app.

The transaction lifecycle, on screen

BEFORE SUBMIT

- Idle
- Preparing

IN FLIGHT

- Signing
- Submitting
- Confirming

AFTER RPC

- Confirmed
- **Expired**
- Error

Each state has its own copy, its own color, its own call-to-action. **No generic "Loading…"**.

Expiry is not an error

Blockhashes are valid roughly **60–90 seconds**. If the transaction isn't included, it expires.

BAD UX

- Silent retry that loops forever.
- "Something went wrong." toast.
- User has no idea if funds moved.

GOOD UX

- Dedicated **expired** state.
- CTA: "Retry with fresh blockhash".
- Preserves the form so the user only re-signs.

Source: Confirmation guide

Priority fees, in plain language

- Priority fees buy **compute-unit priority** during congestion.
- Measured in **micro-lamports per compute unit**.
- They are **additional** to the base network fee.

IN THE UI

- Off / Auto / Manual
- One-line explanation
- Final cost visible **before** signing.

```
import { ComputeBudgetProgram } from "@solana/web3.js";

const ix = ComputeBudgetProgram.setComputeUnitPrice({
  microLamports: 5_000,
});
```


Source: Priority fees

Mobile wallets — the honest version

SUPPORTED TODAY

- **Android**, Chrome on Android, via Mobile Wallet Adapter (MWA).
- Works with major Android wallets.

NOT YET SUPPORTED

- **iOS** has no official MWA support.
- Do not fake it — show a factual "mobile connection is desktop-only right now" message.

> Lying about mobile support is a fast way to lose a user forever.

Source: docs.solanamobile.com

HANDS ON

You will ship a real SOL transfer on Devnet, with the full lifecycle exposed to the user.

Assignment 2 — live recap

- Install the official Wallet Adapter React packages.
- Wrap the app in the three providers.
- Build a transfer form with recipient + amount.
- Surface **every** lifecycle state.
- Add a Solana Explorer link after confirmation.

Full spec: `assignments/assignment-2.md`.

Quiz 2 — topics covered

- The three providers and their responsibilities.
- Why pre-signature explanation builds trust.
- How to handle expired transactions gracefully.
- Priority fee modes and user-facing UX.
- Real MWA support on Android vs iOS.

> Full quiz with answers: `quizzes/quiz-2.md`.

END OF LECTURE 02

Next: Production UI patterns.

RPC failover, accessibility, priority fee ergonomics, and the difference between "works on my localhost" and "ready for Mainnet".

References

- Wallet Adapter — github.com/anza-xyz/wallet-adapter
- Frontend docs — solana.com/docs/frontend
- Confirmation — solana.com/developers/guides/advanced/confirmation
- Priority fees — solana.com/developers/guides/advanced/how-to-use-priority-fees
- Solana Mobile — docs.solanamobile.com